

Introduction

Low-Level Design (LLD) interviews evaluate your ability to transform real-world scenarios into clean, scalable, and easy-to-maintain code structures. One of the most commonly discussed examples is the **Elevator System LLD Design**.

In this article, we'll guide you through a clear step-by-step framework to design this system, just like you would in a real interview. Along the way, we'll share pro tips to help you articulate your reasoning, handle edge cases, and impress your interviewer with a well-thought-out approach.

Step 1: Clarify Requirements

Functional Requirements:

- Building should support multiple floors and multiple elevators
- Each floor should have external panels with up/down buttons
- Each elevator should have internal panels with floor selection, open, close buttons
- System should dispatch elevators efficiently to minimize response time
- Elevators should move based on dynamic strategies (FCFS, SCAN)
- People can enter/exit only when elevator is stopped with doors open
- System should support adding/removing elevators dynamically
- System should maintain building constraints (top floor, bottom floor)

Interview Tip: Always ask the interviewer to confirm the requirements before jumping into code. It shows clarity and alignment.

Edge Cases to Consider:

- What happens when elevator reaches capacity? What is the capacity?
 - How to handle emergency situations?
 - What if multiple elevators are equidistant from a request?
 - How to handle elevator maintenance mode?
-

Step 2: Identify Core Entities

1. Building

- id: String [PK]
- name: String
- minFloor: int
- maxFloor: int
- totalElevators: int
- systemState: SystemState (RUNNING, STOPPING, STOPPED, MAINTENANCE)

2. Elevator

- id: String [PK]
- buildingId: String [FK]
- currentFloor: int
- direction: Direction (UP, DOWN, IDLE)
- capacity: int
- currentLoad: int
- isActive: boolean
- stateHandler: ElevatorStateHandler (Holds the current state)

3. ExternalRequest

- id: String [PK]
- floorNumber: int
- buildingId: String [FK]
- direction: Direction (UP, DOWN)
- timestamp: long
- status: RequestStatus (PENDING, ASSIGNED, COMPLETED, QUEUED)
- assignedElevatorId: String [FK] (nullable)

4. InternalRequest

- id: String [PK]
- elevatorId: String [FK]
- destinationFloor: int
- timestamp: long
- status: RequestStatus (PENDING, COMPLETED)

5. Direction (Enum)

- UP
- DOWN
- IDLE

6. ElevatorState (Enum)

- MOVING
- STOPPED
- DOORS_OPENING
- DOORS_CLOSING
- MAINTENANCE

7. RequestStatus (Enum)

- PENDING
- ASSIGNED
- COMPLETED
- QUEUED

8. SystemState (Enum)

- RUNNING
- STOPPING
- STOPPED
- MAINTENANCE

Interview Tip: Defining entities like Building, Elevator, and Requests early ensures you can model both internal and external interactions. Clearly capturing states (Direction, ElevatorState, SystemState) is crucial for handling real-world scenarios like load, failures, and maintenance.

Step 3: Visualize Interaction Flows

1. Elevator Operations Flow:

- Create new elevator in building with given capacity
- Move elevator to target floor when needed
- Set elevator to maintenance mode when required
- Admin starts elevator system
- Admin stops elevator system

2. Floor Panel Flow (External Requests):

- User presses up button on floor panel
- User presses down button on floor panel

3. Elevator Panel Flow (Internal Requests):

- User inside elevator selects destination floor
-

Step 4: Define Class Structures & Relationships

Architecture Layers

Client/UI → Controller Layer → Service Layer → Domain Layer

CONTROLLERS:

1. ElevatorController (Elevator Operations)

- - Elevator createElevator(String buildingId, int capacity)
- - void moveElevator(String elevatorId, int targetFloor) // specific request
- - void setElevatorMaintenance(String elevatorId, boolean maintenance)
- - void startElevatorSystem(String buildingId)
- - void stopElevatorSystem(String buildingId)

2. FloorPanelController (External Requests)

- - void pressUpButton(int floorNumber, String buildingId)
- - void pressDownButton(int floorNumber, String buildingId)

3. ElevatorPanelController (Internal Requests)

- - void selectFloor(String elevatorId, int destinationFloor)

SERVICES:

1. ElevatorService (Elevator Management)

- - Elevator createElevator(String buildingId, int capacity)
- - void updateElevatorState(String elevatorId, ElevatorState state)

- - void updateElevatorFloor(String elevatorId, int floor)
 - - List<Elevator> getAvailableElevators(String buildingId)
 - - Elevator findById(String elevatorId)
 - - List<Elevator> findByBuilding(String buildingId)
 - - void setRequestService(RequestService requestService) //
- Dependency injection

2. RequestService (Request Processing)

- - ExternalRequest createExternalRequest(int floor, Direction direction, String buildingId)
- - InternalRequest createInternalRequest(String elevatorId, int destinationFloor)
- - void completeRequest(String requestId)

3. DispatcherService (Elevator Selection & Request Queuing)

- - void queueExternalRequest(ExternalRequest request)
- - Elevator selectBestElevator(ExternalRequest request, List<Elevator> availableElevators)
- - void assignRequestToElevator(ExternalRequest request, Elevator elevator)
- - void processPendingRequests(String buildingId)
- - void setElevatorSelectionStrategy(ElevatorSelectionStrategy strategy)

4. MovementService (Elevator Movement & Scheduling)

- - void processElevatorMovement(String elevatorId, Elevator elevator)
- - void processAllElevatorMovements(String buildingId)
- - List<Integer> calculateStops(Elevator elevator, List<InternalRequest> requests)
- - void calculateStops(String elevatorId)
- - void setMovementStrategy(MovementStrategy strategy)
- - void startElevatorSystem(String buildingId) // Initializes ScheduledExecutorService

- - void stopElevatorSystem(String buildingId) // Gracefully shuts down scheduler
- - boolean hasPendingRequests(String buildingId) // Helper for graceful shutdown

5. BuildingService (Building Management & System Control)

- - Building createBuilding(String name, int minFloor, int maxFloor, int totalElevators)
 - - boolean isValidFloor(String buildingId, int floor)
 - - void setBuildingSystemState(String buildingId, SystemState state)
 - - boolean isSystemRunning(String buildingId)
 - - Building findById(String buildingId)
 - - boolean buildingExists(String buildingId)
-

REPOSITORIES:

1. ElevatorRepository

- - Elevator save(Elevator elevator)
- - Optional<Elevator> findById(String elevatorId)
- - List<Elevator> findByBuilding(String buildingId)
- - List<Elevator> findAvailableElevators(String buildingId)
- - void deleteById(String elevatorId)

2. ExternalRequestRepository

- - ExternalRequest save(ExternalRequest request)
- - List<ExternalRequest> findPendingRequests(String buildingId)
- - List<ExternalRequest> findQueuedRequests(String buildingId)
- - void updateRequestStatus(String requestId, RequestStatus status)
- - Optional<ExternalRequest> findById(String requestId)
- - List<ExternalRequest> findAll()
- - void deleteById(String requestId)

3. InternalRequestRepository

- - InternalRequest save(InternalRequest request)
- - List<InternalRequest> findByElevator(String elevatorId)
- - List<InternalRequest> findPendingByElevator(String elevatorId)
- - Optional<InternalRequest> findById(String requestId)
- - List<InternalRequest> findAll()
- - void updateRequestStatus(String requestId, RequestStatus status)

4. BuildingRepository

- - Building save(Building building)
- - Optional<Building> findById(String buildingId)
- - List<Building> findAll()
- - void deleteById(String buildingId)

STRATEGY PATTERN IMPLEMENTATION:

1. ElevatorSelectionStrategy (Strategy Interface)

- + Elevator selectElevator(ExternalRequest request, List<Elevator> elevators)

2. NearestElevatorStrategy (Concrete Strategy)

- + Elevator selectElevator() // Select closest available elevator

3. LoadBalancingStrategy (Concrete Strategy)

- + Elevator selectElevator() // Select elevator with least load

4. MovementStrategy (Strategy Interface)

- + List<Integer> calculatePath(Elevator elevator, List<InternalRequest> requests)

5. ScanStrategy (Concrete Strategy)

- + List<Integer> calculatePath() // SCAN algorithm - continue in current direction

6. FCFSStrategy (Concrete Strategy)

- + List<Integer> calculatePath() // First Come First Serve
-

STATE PATTERN IMPLEMENTATION:

1. ElevatorStateHandler (State Interface)

- + void openDoors(Elevator elevator)
- + void closeDoors(Elevator elevator)
- + void enterMaintenance(Elevator elevator)
- + void exitMaintenance(Elevator elevator)
- + boolean canAcceptExternalRequests(Elevator elevator)
- + boolean canAcceptInternalRequests(Elevator elevator)
- + String getStateName()

2. MovingState (Concrete State)

- + void openDoors() // Cannot open doors while moving
- + void closeDoors() // Already closed while moving
- + void enterMaintenance() // Transition to PreMaintenanceState

3. StoppedState (Concrete State)

- + void openDoors() // Change to DoorsOpeningState
- + void closeDoors() // Already closed
- + void enterMaintenance() // Change to MaintenanceState

4. DoorsOpeningState (Concrete State)

- + void openDoors() // Already opening
- + void closeDoors() // Change to DoorsClosingState

5. DoorsClosingState (Concrete State)

- + void openDoors() // Change to DoorsOpeningState
- + void closeDoors() // Already closing

6. MaintenanceState (Concrete State)

- + void openDoors() // Allow for maintenance access
- + void closeDoors() // Allow for maintenance access
- + void exitMaintenance() // Change back to StoppedState

7. PreMaintenanceState

- + void enterMaintenance() // Already transitioning to maintenance
 - + void exitMaintenance() // Cancel maintenance transition
-

Step 5: Core Use Cases & Methods

1. External Request Use Case:

- POST /api/floors/{floorNumber}/request → FloorPanelController.pressUpButton()
 - RequestService.createExternalRequest() → DispatcherService.queueExternalRequest()
 - Return confirmation (request queued)
-

2. Internal Request Use Case:

- POST /api/elevators/{elevatorId}/floors → ElevatorPanelController.selectFloor()
- RequestService.createInternalRequest()
- Return confirmation (request queued)

3. Elevator Creation Use Case:

- POST /api/elevators → ElevatorController.createElevator()
- ElevatorService.createElevator()
- Validate building constraints → Create elevator → Return elevator

4. System Start Use Case:

- POST /api/elevators/start →
ElevatorController.startElevatorSystem()
- MovementService.startElevatorSystem() →
DispatcherService.processPendingRequests()
- Process all queued requests using strategy pattern → Return confirmation

5. System Stop Use Case:

- POST /api/elevators/stop →
ElevatorController.stopElevatorSystem()
- MovementService.stopElevatorSystem()
- Complete pending requests gracefully → Set system state to STOPPING → Reject new requests → Return confirmation

6. Elevator Movement Use Case (Background Process):

- MovementService.processAllElevatorMovements()

- Get pending requests → Apply movement strategy → Calculate next floor
 - Update elevator position → Check if stop required → Open/close doors
-

Step 6: Apply OOP Principles & Design Patterns

Design Patterns Used:

- **Strategy Pattern** – for elevator selection and movement algorithms
 - **State Pattern** – for elevator door and maintenance states
 - **Repository Pattern** – for data access abstraction
 - **Service Layer Pattern** – for business logic separation
 - **Scheduler Pattern** – for periodic background processing using `ScheduledExecutorService`
-

OOP Principles Applied:

- **Single Responsibility** – each service handles specific elevator operations
 - **Open/Closed** – easy to add new selection/movement strategies
 - **Encapsulation** – elevator state and behavior encapsulated in domain objects
 - **Dependency Inversion** – services depend on strategy interfaces, not implementations
 - **Polymorphism** – different strategies can be used interchangeably
-

Step 7: Handle Edge Cases

Edge Case Solutions:

- **Elevator reaches capacity** – Track currentLoad vs capacity in Elevator entity; skip floors when full; display "Full" on external panels; dispatcher assigns another elevator
 - **Emergency situations** – Add emergency state in ElevatorState enum; emergency button overrides requests; elevator goes to ground floor; others halt until cleared
 - **Multiple elevators equidistant** – Apply secondary criteria (load balancing, elevator ID); LoadBalancingStrategy considers request queue size; ensures deterministic, predictable selection
 - **Elevator maintenance mode** – Set isActive=false and state=MAINTENANCE; finish current requests but block new external ones; dispatcher excludes maintenance elevators; internal requests processed until empty
 - **System start/stop gracefully** – On start: process queued requests with strategy pattern; on stop: finish pending requests, reject new ones; track system state; background scheduler continues until completion
-

Implementation Strategies:

- **Strategy Pattern** – Pluggable algorithms for elevator selection and movement
- **State Management** – Explicit transitions for door, emergency, and maintenance states
- **Request Queuing** – Thread-safe external requests via BlockingQueue
- **Load Balancing** – Distribute requests across elevators with LoadBalancingStrategy
- **Capacity Management** – Enforce load limits via isFull() method

- **Maintenance Support** – Graceful handling of downtime with maintenance states
 - **System Control** – Controlled start/stop with graceful request processing
 - **Repository Pattern** – Abstract data with in-memory concurrent structures
 - **Scheduler Pattern** – Background processing via `ScheduledExecutorService`
 - Request Processing: 1-second intervals for queue handling
 - Elevator Movement: 2-second intervals for movement & updates
 - Thread Pool: 2 threads for concurrent scheduling tasks
 - Graceful Shutdown: System waits for pending requests before stop
-

Summary

The Elevator System is designed to efficiently manage multiple elevators with a focus on reliability, scalability, and user safety. It handles edge cases such as capacity limits, emergency overrides, equidistant elevator conflicts, and maintenance downtime.

The system leverages the **Strategy Pattern** for flexible elevator selection, **State Management** for clear transitions, and **BlockingQueues** for thread-safe request handling. Load balancing ensures fair distribution of requests, while schedulers manage background tasks like movement and request processing.

Maintenance and graceful start/stop mechanisms allow continuous, fault-tolerant operation.